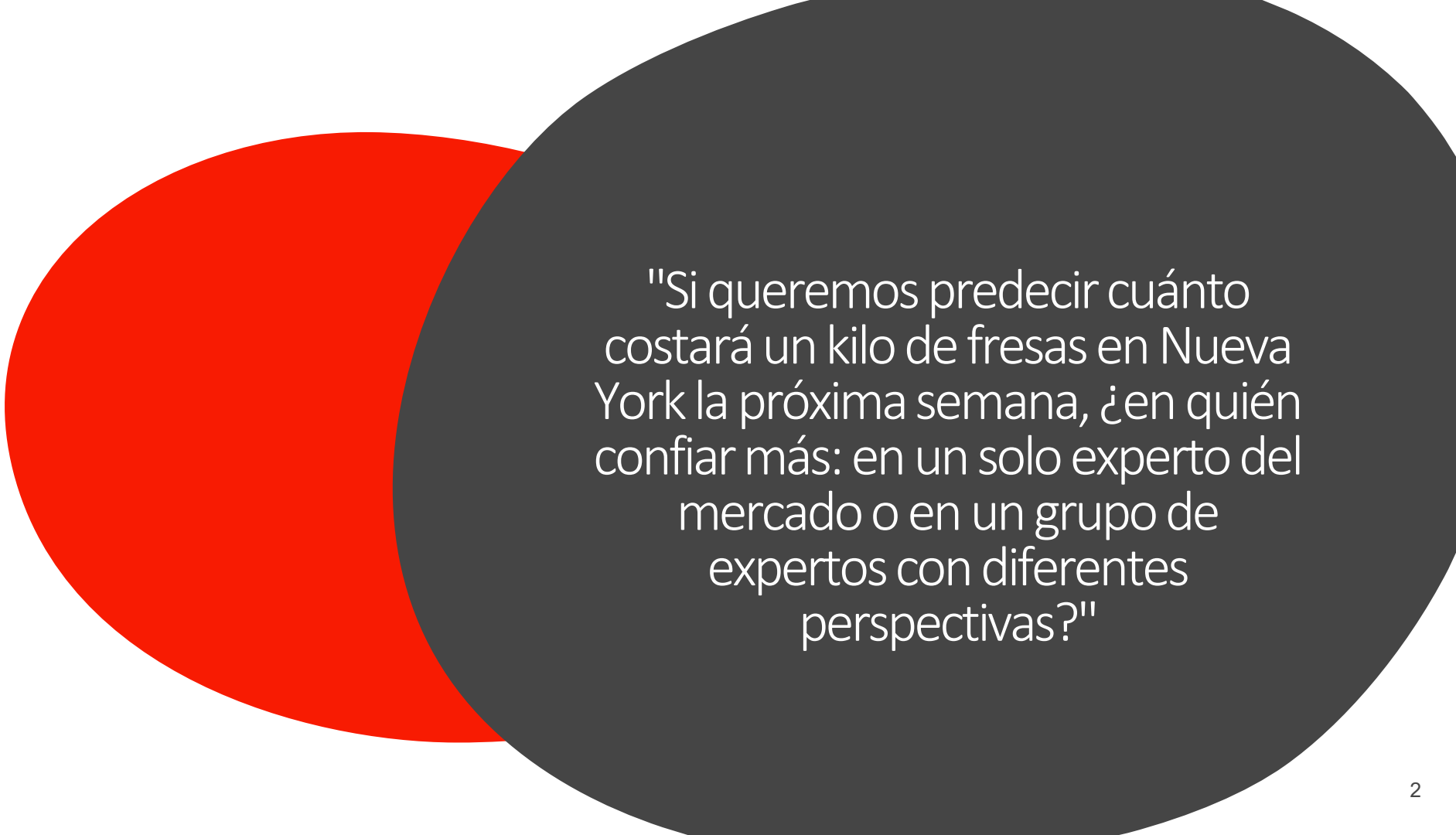
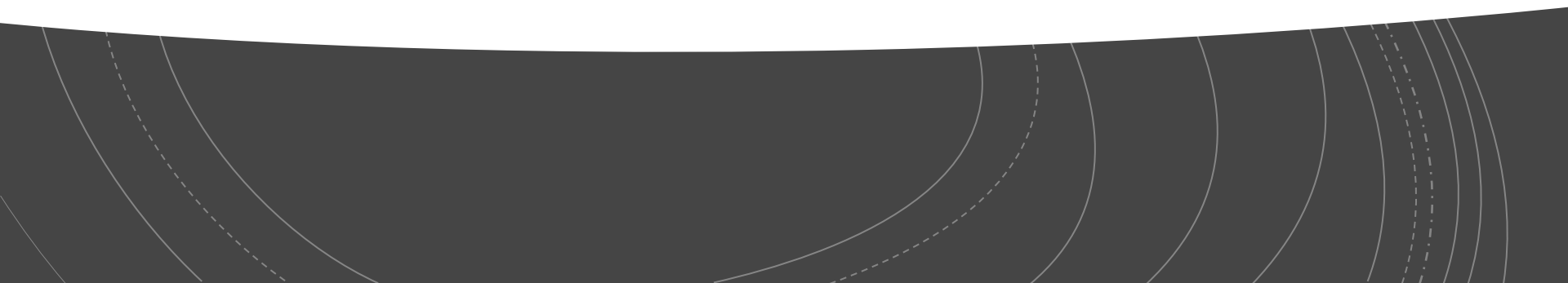


RANDOM FOREST



"Si queremos predecir cuánto costará un kilo de fresas en Nueva York la próxima semana, ¿en quién confiar más: en un solo experto del mercado o en un grupo de expertos con diferentes perspectivas?"

“Tenemos un algoritmo llamado Random Forest, que funciona exactamente así: en lugar de confiar en una sola opinión (un árbol de decisión), crea un grupo de expertos (un bosque) para tomar mejores decisiones.”



- Queremos saber **cuál será el precio de las fresas la próxima semana en Nueva York**. No se tiene una bola de cristal, pero decides preguntarle a varias personas que conocen sobre mercados agrícolas:
- **El Agricultor:** Te dice que el precio en la granja es bajo, así que el precio en la ciudad también debería ser bajo.
- **El Vendedor de Atlanta:** Te dice que si en Atlanta las fresas están caras, es probable que también lo estén en Nueva York.
- **El Comerciante de Chicago:** Basado en su experiencia, dice que si en Chicago las fresas subieron de precio, lo mismo podría pasar en Nueva York.
- **El Distribuidor de Los Ángeles:** Analiza tendencias y dice que el clima afectará los precios la próxima semana.





Cada persona te da una estimación basada en su conocimiento. En lugar de confiar en una sola persona, decides **tomar el promedio de todas sus predicciones** para obtener una estimación más confiable.



Esto es exactamente lo que hace **Random Forest**:

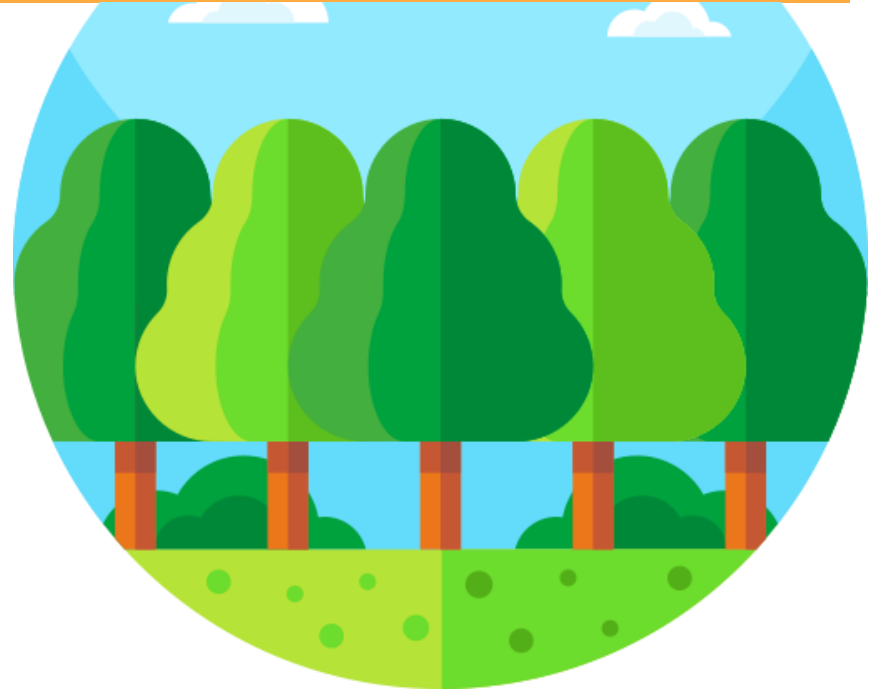
- ✓ Reúne varias "opiniones" (árboles de decisión).
- ✓ Cada árbol analiza diferentes aspectos de los datos.
- ✓ Al combinar todas las predicciones, obtenemos un resultado más preciso y robusto.

En resumen: en lugar de confiar en un solo experto (un árbol de decisión), confiamos en un equipo de expertos (un bosque de árboles). 🌳 🔥

<https://www.youtube.com/watch?v=tYPi6qcCQbg>

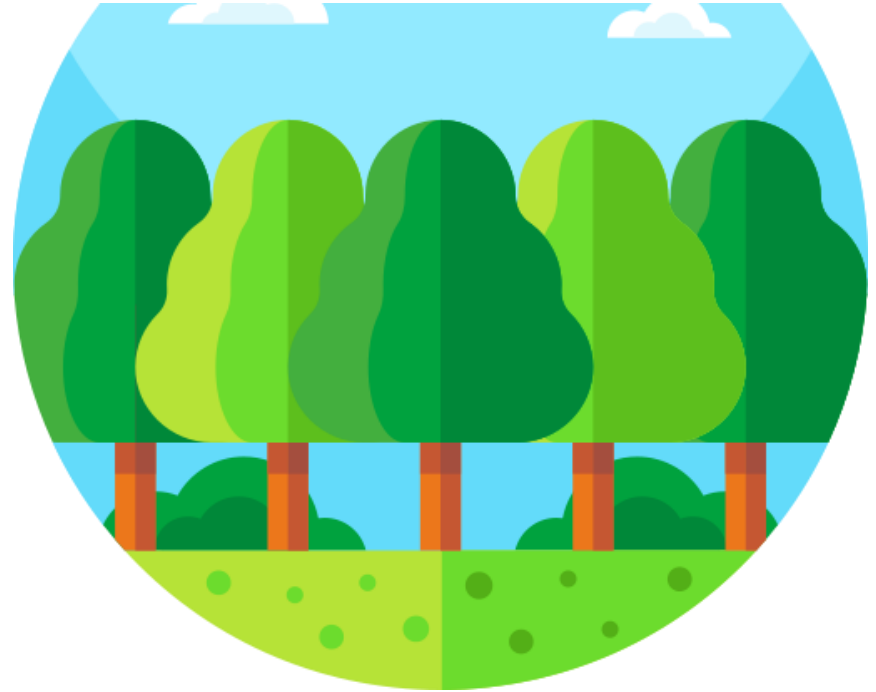
BOSQUES ALEATORIOS

Random Forest es un tipo de Ensamble en Machine Learning en donde combinaremos diversos árboles de decisión. Pero entonces, ¿Qué son los métodos de Ensamble en ML?



MÉTODOS DE ENSAMBLE DE MODELOS

También métodos combinados , intentan ayudar a mejorar el rendimiento de los modelos de Machine Learning. Este es un proceso mediante el cual se construyen estratégicamente varios modelos de ML para resolver un problema particular.



◆ Principales enfoques de Ensemble

**Bagging
(Bootstrap
Aggregating)**

Boosting

**Stacking
(Stacked
Generalization)**

Bagging (Bootstrap Aggregating)

Objetivo: Reducir la varianza del modelo combinando múltiples predictores entrenados en subconjuntos aleatorios del conjunto de datos.

Modelos típicos:

- Bagged Decision Trees
- Bagged KNN
- Random Forest
- Bagged SVMs

Boosting

Objetivo: Reducir el sesgo del modelo al entrenar secuencialmente modelos que corrigen los errores del anterior.

Modelos típicos:

- AdaBoost
- Gradient Boosting Machines (GBM)
- XGBoost
- LightGBM
- CatBoost
- LogitBoost

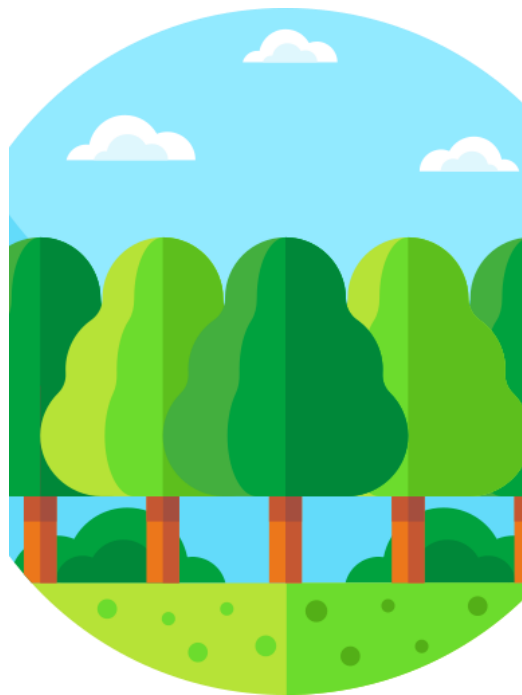
Stacking (Stacked Generalization)

Objetivo: Combinar múltiples modelos (de diferentes tipos) usando un modelo meta que aprende a partir de sus predicciones.

Modelos típicos:

- **Stacked Ensemble Models** (pueden combinar regresores, clasificadores, árboles, SVM, etc.)
- **Modelos Meta** (como una regresión logística o red neuronal) usados para combinar modelos base

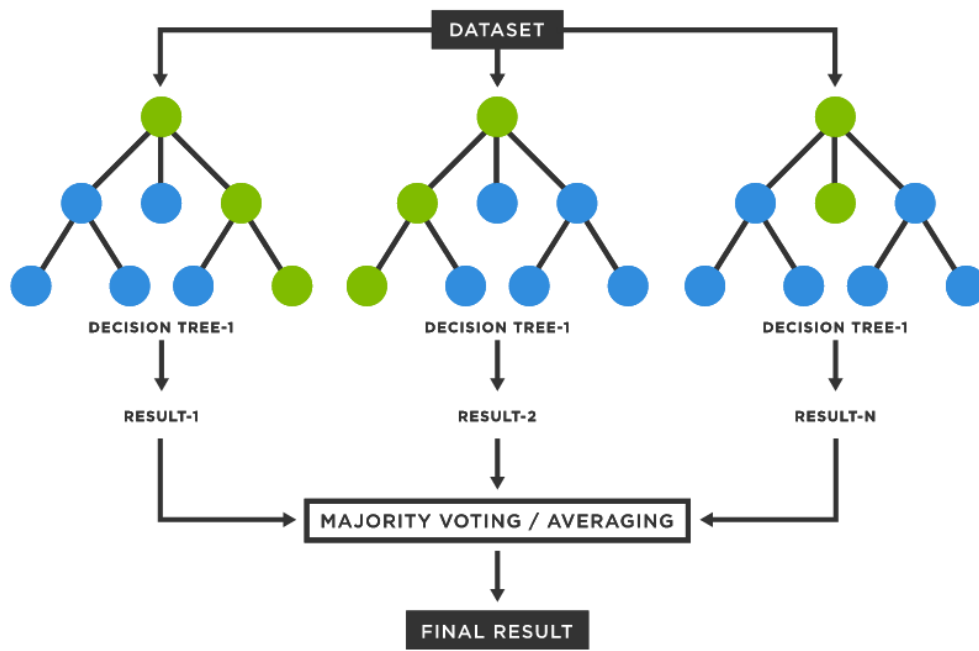
¿CÓMO SURGE RANDOM FOREST?



Uno de los problemas que aparecía con la creación de un árbol de decisión, es que si le damos la profundidad suficiente, el árbol tiende a “memorizar” las soluciones en vez de generalizar el aprendizaje. Es decir, a padecer de overfitting. La solución para evitar esto es la de crear muchos árboles y que trabajen en conjunto.



BOSQUES ALEATORIOS



¿Qué es Random Forest?



Random Forest es un algoritmo de **aprendizaje supervisado** que se utiliza tanto para clasificación como para regresión. Se basa en la combinación de múltiples árboles de decisión para mejorar la precisión y reducir el sobreajuste (*overfitting*).



Podemos imaginar Random Forest como un grupo de expertos en mercados agrícolas. Cada experto (árbol de decisión) hace una predicción sobre el precio de un producto agrícola basándose en datos como el precio en la granja y los precios en otras ciudades. Luego, combinamos las respuestas de todos los expertos para obtener una predicción más precisa.

¿Cómo funciona Random Forest?



**Muestreo Aleatorio de
Datos (*Bootstrap
Sampling*)**



**Creación de Múltiples
Árboles de Decisión**

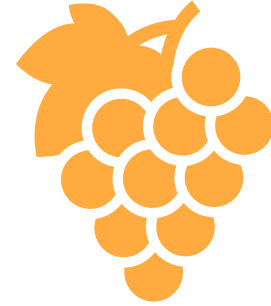


**Predicción y
Agregación de
Resultados**

1. Muestreo Aleatorio de Datos (*Bootstrap Sampling*):



Se generan múltiples subconjuntos del dataset original tomando datos aleatorios con reemplazo (lo que significa que algunos datos pueden repetirse en un subconjunto).



En nuestro caso, podríamos tomar subconjuntos de productos como naranjas, fresas y lechugas, con diferentes precios en cada ciudad.

2. Creación de Múltiples Árboles de Decisión:



Cada subconjunto se usa para entrenar un árbol de decisión independiente.



Un árbol de decisión es un modelo basado en preguntas binarias como:

"¿El precio en la granja es mayor a \$1.00?"
"¿El precio en Los Ángeles es más bajo que en Chicago?"



Cada árbol aprenderá diferentes patrones dependiendo de los datos que reciba.

3. Predicción y Agregación de Resultados:



PARA REGRESIÓN,
EL RESULTADO
FINAL ES EL
PROMEDIO DE LAS
PREDICCIONES DE
TODOS LOS
ÁRBOLES.



PARA
CLASIFICACIÓN,
SE TOMA EL
VOTO
MAYORITARIO
(EL RESULTADO
QUE MÁS
ÁRBOLES
ELIGEN).

En nuestro caso, si estamos prediciendo el precio en Nueva York, cada árbol dará una estimación basada en los precios en la granja y en otras ciudades. Luego, Random Forest tomará el promedio de esas predicciones para obtener el resultado final.

Ventajas de Random Forest

Reducción del
sobreajuste
(*Overfitting*)

Alta Precisión

Manejo de
Datos
Ruidosos o
Faltantes

Importancia de
Características

Funciona con
Grandes
Volúmenes de
Datos

Ventajas de Random Forest

Reducción del sobreajuste (*Overfitting*)

Un solo árbol de decisión puede ajustarse demasiado a los datos y generar predicciones inexactas.

Al combinar varios árboles, Random Forest reduce este problema, lo que lo hace más **robusto y preciso**.

Alta Precisión

Es uno de los algoritmos más precisos porque combina múltiples modelos.

En nuestro dataset de precios de frutas, nos ayuda a obtener predicciones más confiables.

Ventajas de Random Forest

Manejo de Datos Ruidosos o Faltantes

- Puede manejar datos con valores faltantes y **es menos sensible a datos atípicos**.
- Por ejemplo, si un día los precios de fresas en Chicago son extremadamente altos, el modelo no se verá afectado porque otros árboles equilibrarán la predicción.

Importancia de Características

- Puede indicar qué variables tienen mayor impacto en la predicción.
- En nuestro caso, podríamos ver si el **precio en la granja** influye más que el **precio en otras ciudades**.

Funciona con Grandes Volúmenes de Datos

- Puede manejar grandes cantidades de datos y muchas características sin perder precisión.

Desventajas de Random Forest

Puede ser más lento

- Debido a que entrena muchos árboles, puede ser más lento en comparación con modelos más simples.
- Sin embargo, esto no es un problema si los datos no son enormes.

Difícil de Interpretar

- Un solo árbol de decisión es fácil de entender, pero cuando combinamos cientos de árboles, el modelo se vuelve una “caja negra” difícil de explicar.

Ejemplo con Dataset (Precios de Frutas)

Imaginemos que queremos predecir el precio de las naranjas en Nueva York. Supongamos que tenemos los siguientes datos:

Precio en la Granja	Precio en Atlanta	Precio en Chicago	Precio en Los Ángeles	Precio en Nueva York
\$0.50	\$1.20	\$1.40	\$1.10	\$1.50
\$0.55	\$1.30	\$1.35	\$1.25	\$1.60
\$0.60	\$1.25	\$1.50	\$1.20	\$1.55

Cada árbol de decisión en el bosque tomará diferentes subconjuntos de estos datos y hará reglas como:

- "Si el precio en la granja es mayor a \$0.55, el precio en Nueva York es aproximadamente \$1.60".
- "Si el precio en Atlanta es menor a \$1.25, el precio en Nueva York es aproximadamente \$1.50".

Después de hacer esto en muchos árboles, tomamos el promedio de las predicciones y obtenemos el precio final.



Random Forest es una herramienta poderosa para hacer predicciones en escenarios donde hay múltiples variables interrelacionadas. En este caso, nos permite predecir con precisión los precios de productos agrícolas en diferentes mercados basándose en patrones aprendidos de datos históricos.

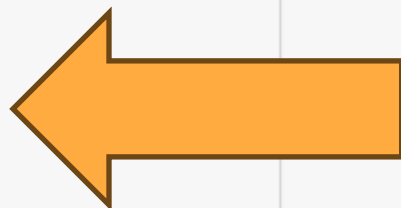
```
# =====
# 3 DIVISIÓN DE DATOS
# =====
# Variables independientes (X) y variable objetivo (y)
X = df[['farmprice', 'atlantaretail', 'chicagoretail', 'losangelesretail']]
y = df['newyorkretail']

# Dividir en entrenamiento (80%) y prueba (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# =====
# 4 ENTRENAR RANDOM FOREST
# =====
# Modelo de Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Predicciones
y_pred = rf.predict(X_test)

# Evaluación del modelo
mae = mean_absolute_error(y_test, y_pred)
print(f"\n🔍 MAE del modelo: {mae:.2f}")
```



Aquí estamos creando un **Random Forest Regressor**, que es un conjunto de árboles de decisión que trabajarán juntos para hacer predicciones.

Parámetros clave:

n_estimators=100: Esto significa que el bosque tendrá **100 árboles de decisión**. Más árboles pueden mejorar la precisión, pero también aumentan el tiempo de cálculo.

random_state=42: Esto fija la aleatoriedad para que los resultados sean **reproducibles** cada vez que ejecutes el código.

```
"""  
# Modelo de Random Forest  
rf = RandomForestRegressor(n_estimators=100, random_state=42)
```

Aquí le estamos diciendo al modelo que aprenda a predecir el precio en Nueva York (`y_train`) usando las variables independientes (`X_train`), que incluyen:

`farmprice`: Precio en la granja

`atlantaretail`: Precio en Atlanta

`chicagoretail`: Precio en Chicago

`losangelesretail`: Precio en Los Ángeles

```
rf.fit(X_train, y_train)
```

Lo que hace internamente:

1. Toma los datos de `X_train` y `y_train`.
2. Crea 100 árboles de decisión diferentes (porque `n_estimators=100`).
3. Cada árbol toma una muestra aleatoria de los datos y aprende a hacer predicciones.
4. Cuando le pidamos una predicción, cada árbol dará su resultado, y el modelo tomará el promedio de todas las predicciones para obtener el valor final.

Imaginemos que queremos predecir el precio de una manzana en Nueva York.

En lugar de preguntarle a una sola persona, preguntamos a **100 expertos** en frutas de diferentes lugares.

- Cada experto (árbol de decisión) da su **mejor estimación** basada en su experiencia.

- El modelo **promedia las respuestas** de todos los expertos para obtener un valor más preciso.

- Si un experto se equivoca, no importa tanto, porque hay muchos otros ayudando a mejorar la predicción.

```
# =====  
# 5 IMPORTANCIA DE VARIABLES  
# =====  
# Obtener importancia de cada variable  
importances = rf.feature_importances_  
feature_names = X.columns
```

rf.feature_importances_ devuelve un **array numérico** con la importancia de cada variable en la predicción.

X.columns devuelve el nombre de las columnas en X, que son las variables independientes usadas para predecir el precio en Nueva York.

rf.feature_importances_ devuelve un **array numérico** con la importancia de cada variable en la predicción.

Cada número representa **qué tan útil** fue una variable en los árboles de decisión dentro del Random Forest.

Las variables más importantes tienen valores más altos.



Ejemplo de salida posible:

```
importances = [0.45, 0.25, 0.15, 0.15]
```

Esto significaría que:

- **farmprice** explica **45%** de la predicción.
- **atlantaretail** aporta **25%**.
- **chicagoretail** y **losangelesretail** tienen menor impacto (**15% cada una**).

X.columns devuelve el nombre de las columnas en X, que son las **variables independientes** usadas para predecir el precio en Nueva York. Esto nos permite asociar cada importancia con su respectiva variable.

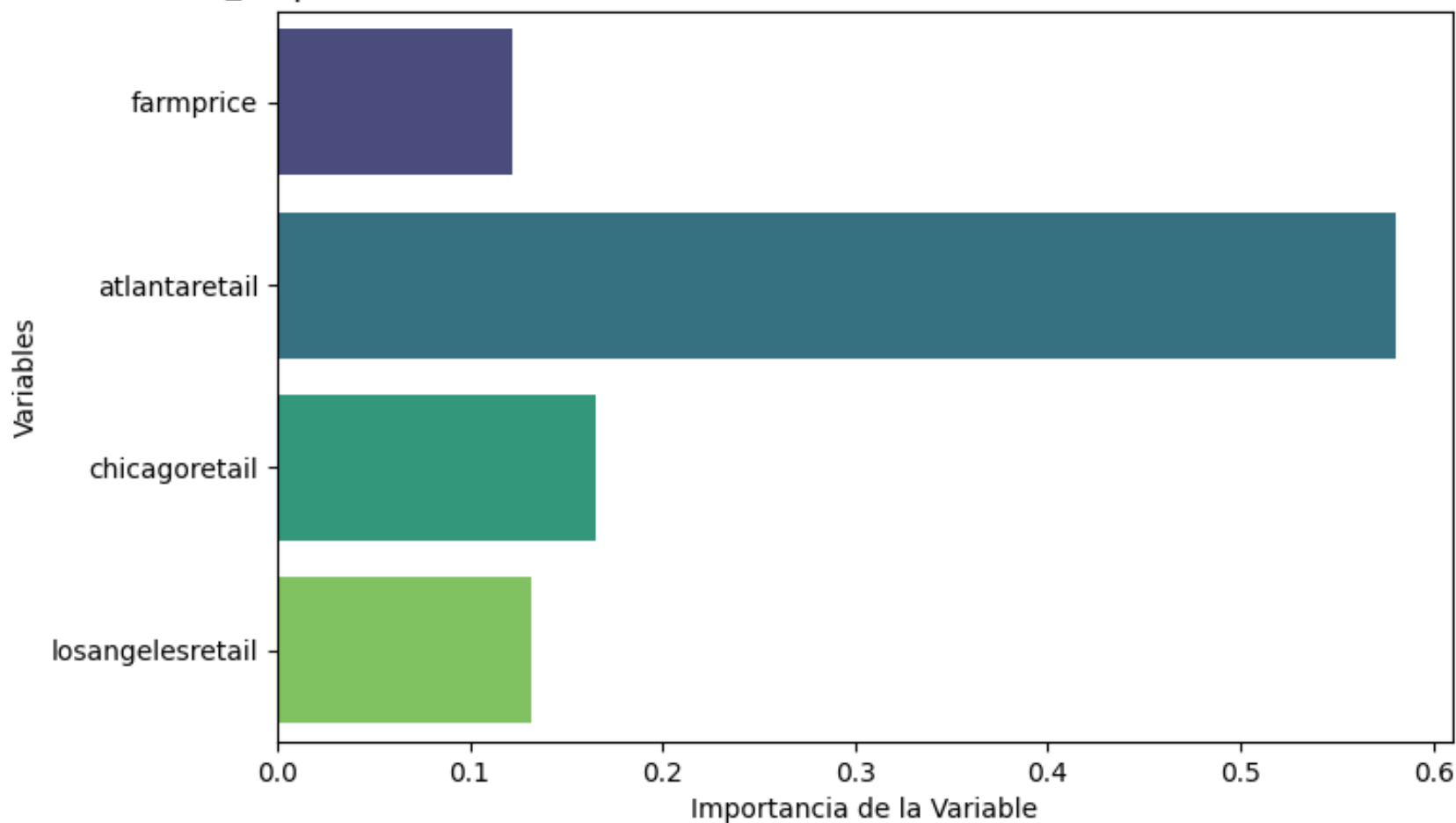
Ejemplo de salida de **feature_names:**

```
Index(['farmprice', 'atlantaretail', 'chicagoretail', 'losangelesretail'], dtype='object')
```

Ahora podemos hacer un **gráfico** para visualizar qué variables influyen más en la predicción.

```
# Gráfico de importancia de variables
plt.figure(figsize=(8, 5))
#sns.barplot(x=importances, y=feature_names, palette="viridis")
sns.barplot(x=importances, y=feature_names, hue=feature_names, legend=False, palette="viridis")
```

□ Importancia de Variables en la Predicción de Precios en Nueva York (MAE: 0.33)



¿Qué representa cada barra?

Cada barra representa **cuánto contribuye** una variable a mejorar la predicción del precio en Nueva York. Cuanto más larga sea la barra, más importante es esa variable para el modelo.

En este caso:

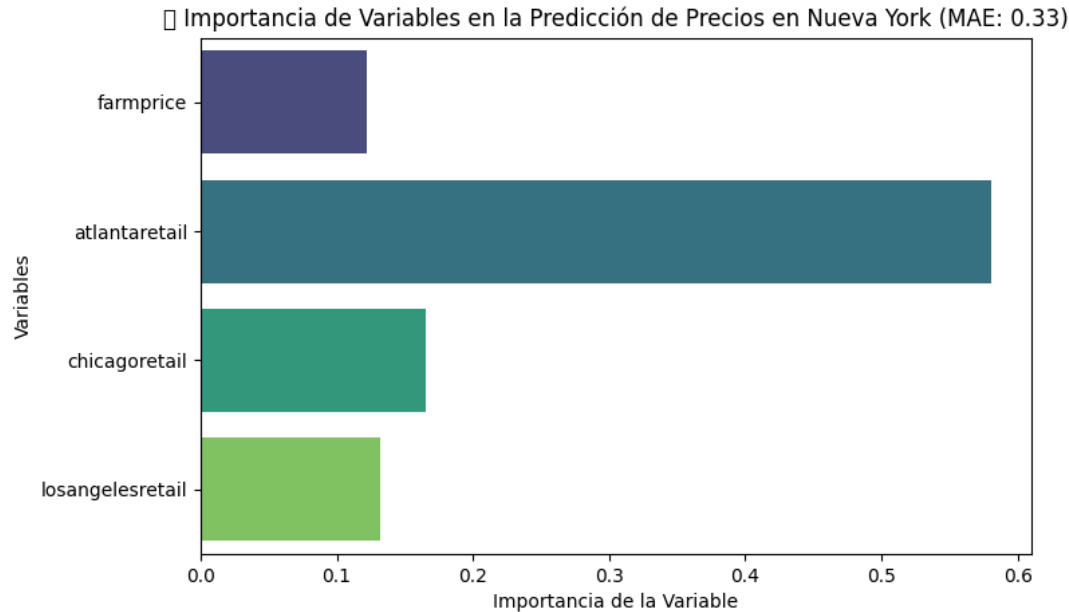
- ❶ **atlantaretail** → Es la variable más importante con diferencia (~ 0.6), lo que significa que el precio en Atlanta tiene la mayor influencia en el precio en Nueva York.
- ❷ **chicagoretail y losangelesretail** → También tienen influencia (~ 0.2 y ~ 0.15 respectivamente), pero menor que **atlantaretail**.
- ❸ **farmprice** → Tiene la menor importancia (~ 0.1), lo que indica que el precio en la granja no es tan relevante para predecir el precio en Nueva York.

¿Por qué atlantaretail es tan importante?

Esto podría deberse a que los precios en Atlanta tienen una **correlación más fuerte** con los precios en Nueva York. Quizás porque comparten patrones de distribución, costos de transporte similares o porque sus mercados están más conectados.

¿Cómo usar esta información?

- Si quisiéramos reducir la cantidad de variables en nuestro modelo, podríamos considerar eliminar farmprice, ya que tiene la menor influencia.
- Si atlantaretail es tan importante, podríamos analizar más datos de esa ciudad para mejorar las predicciones.
- Esto nos ayuda a entender **qué factores realmente afectan el precio en Nueva York** y descartar información menos relevante.



En resumen:

- atlantaretail es la variable más influyente.
- chicagoretail y losangelesretail tienen una influencia media.
- farmprice es la variable menos importante.

"¿Por qué **atlantaretail** tiene tanta importancia en la predicción del precio en Nueva York?"

- "¿Creen que el transporte de bienes juega un papel en esto?"
- "¿Han notado si los precios de frutas o verduras en distintas ciudades cambian de forma parecida?"
- "Si Nueva York compra mucha fruta de Atlanta, ¿cómo afectaría esto a los precios?"

"¿Por qué atlantaretail tiene tanta importancia en la predicción del precio en Nueva York?"

- Factores de mercado similares
- Costos de transporte y distribución
- Factores de oferta y demanda regionales
- Patrones históricos o estacionales

◆ Factores de mercado similares

"Atlanta y Nueva York pueden compartir tendencias económicas similares, lo que hace que sus precios se comporten de manera parecida."

Explicación: Si la demanda y oferta en ambas ciudades sigue patrones parecidos, es lógico que los precios también lo hagan.

◆ Costos de transporte y distribución

"El costo de transportar productos desde Atlanta a Nueva York puede influir en los precios finales."

Explicación: Si Nueva York importa productos desde Atlanta, sus precios pueden depender de los de allá.

◆ Factores de oferta y demanda regionales

"Si hay una conexión comercial fuerte entre Atlanta y Nueva York, los cambios de precios en una ciudad pueden reflejarse en la otra."

Explicación: A veces, mercados lejanos están más conectados de lo que pensamos por acuerdos comerciales o costos de producción.

◆ Patrones históricos o estacionales

"Quizás históricamente los precios en Atlanta han sido buenos indicadores del mercado en Nueva York."

Explicación: A veces, patrones de precios pasados pueden predecir tendencias futuras.

2 cosas para mejorar nuestro modelo:

Optimizar Random Forest ajustando hiperparámetros clave.
Visualizar la importancia de las variables para entender cuáles influyen más en la predicción del precio en Nueva York.

Hiperparámetros Clave en Random Forest

n_estimators (Número de árboles)

Es la cantidad de árboles en el bosque.

Más árboles → **Mejor estabilidad**, pero aumenta el tiempo de cómputo.

Demasiados árboles pueden hacer que el modelo sea más lento sin mejorar mucho la precisión.

Estrategia:

✓ **Empieza con 100 árboles** y aumenta gradualmente hasta encontrar el mejor valor.

```
rf = RandomForestRegressor(n_estimators=200, random_state=42)
```

Hiperparámetros Clave en Random Forest

max_depth (Profundidad máxima del árbol)

Controla hasta qué nivel se expande cada árbol.

Mayor profundidad → Aprende más patrones, pero puede sobreajustar (**overfitting**).

Demasiado pequeño → No aprende lo suficiente (**underfitting**).

Estrategia:

✓ **Prueba valores entre 5 y 20** para encontrar el equilibrio.

```
rf = RandomForestRegressor(max_depth=10, random_state=42)
```

Hiperparámetros Clave en Random Forest

min_samples_split (Mínimo de muestras para dividir un nodo)

Controla **cuándo** un nodo puede dividirse en dos.

Valores bajos (ej. 2 o 5) → Más divisiones → Puede sobreajustarse.

Valores altos (ej. 20 o más) → Menos divisiones → Puede subajustarse.

Estrategia:

✓ **Empieza con 5 o 10** y ajusta según el rendimiento.

```
rf = RandomForestRegressor(min_samples_split=10, random_state=42)
```

Hiperparámetros Clave en Random Forest

min_samples_leaf (Mínimo de muestras en una hoja)

Define el número mínimo de muestras que debe tener una hoja en el árbol.

Valores pequeños → Puede capturar demasiado ruido (sobreajuste).

Valores grandes → Hace predicciones más generales.

Estrategia:

✓ **Prueba valores entre 1 y 10.**

```
rf = RandomForestRegressor(min_samples_leaf=4, random_state=42)
```

Hiperparámetros Clave en Random Forest

max_features (Número de variables consideradas en cada división)

Controla **cuántas variables se evalúan en cada división del árbol**.

Valores bajos (ej. "sqrt") → Reduce sobreajuste y mejora la generalización.

Valores altos (ej. "auto") → Usa todas las variables, pero puede sobreajustarse.

Estrategia:

✓ Usa "sqrt" o "log2" para evitar sobreajuste.

```
rf = RandomForestRegressor(max_features="sqrt", random_state=42)
```

Valores por defecto en RandomForestRegressor

Si no los especificamos, se asigna estos valores predeterminados:

PARÁMETRO	VALOR POR DEFECTO	EXPLICACIÓN
n_estimators	100	100 árboles en el bosque
max_depth	None	Crece hasta que todas las hojas tengan menos de min_samples_split muestras
min_samples_split	2	Un nodo se divide si tiene al menos 2 muestras
min_samples_leaf	1	Cada hoja tiene al menos 1 muestra
max_features	"auto" (sqrt(n))	Usa la raíz cuadrada del número de variables para cada división
bootstrap	True	Usa muestreo con reemplazo para construir cada árbol
oob_score	False	No calcula validación con muestras fuera de bolsa
n_jobs	None	Usa un solo núcleo de CPU
verbose	0	No muestra información extra en la ejecución
warm_start	False	No reutiliza árboles de entrenamientos previos

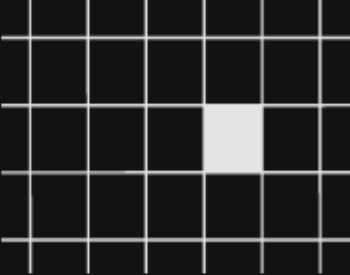
¿Por qué cambiar los valores por defecto?

- **Si el modelo tiene sobreajuste (muy preciso en entrenamiento pero malo en test) →**
 - Reducir `max_depth`, aumentar `min_samples_split` y `min_samples_leaf`.
- **Si el modelo tiene bajo rendimiento (predicciones poco precisas) →**
 - Aumentar `n_estimators`, ajustar `max_features` y `max_depth`.

C1 TALLER

3

- CONTINUANDO CON AREA DE ENTRETANIMIENTO, APLICAR RANDOM FOREST A PROYECTO, TENIENDO EN CUENTA LOS ELEMENTOS DE CLASE.



¡MUCHAS GRACIAS!

